Project Sentinel - Codex Project Brief

Generated for Project Sentinel pre-coding readiness package.

Project Sentinel - Codex Project Brief

One-paragraph summary

Project Sentinel is a modular Cyber Security Operating Environment designed to unify security operations, asset intelligence, identity governance, evidence handling, trust scoring, policy enforcement, workflow automation, telemetry processing, and mission-aware decision support into one coherent engineering platform. The project is designed as an extensible, event-driven security operating system: every asset, identity, event, finding, workflow, policy, risk, piece of evidence, and operational decision is represented as a canonical Sentinel object, moved through a common event bus, linked through a knowledge graph, and evaluated by deterministic engines. The immediate coding objective is to implement the Foundation Runtime first, then build engines incrementally according to the Engineering Archive baseline.

What the project is trying to accomplish

Project Sentinel aims to create a cyber security platform that can operate as a central nervous system for enterprise security. Instead of isolated tools for assets, identity, policy, detection, forensics, workflows, compliance, risk, and AI governance, Sentinel defines a single architecture where these capabilities share a common object model, event model, trust model, evidence model, repository standard, and plugin framework.

The system should eventually support:

- Continuous cyber asset intelligence.
- Identity and machine identity governance.
- Event-driven security automation.
- Evidence-backed findings and decisions.
- Trust-aware policy evaluation.
- Security data lake and telemetry fabric.
- Detection engineering and analytics.
- Attack path and exposure graph analysis.
- Incident command and case management.
- Privacy, data protection, and sovereignty controls.
- AI security and model governance.
- Implementation-ready SDKs, APIs, plugins, tests, and release tooling.

Coding principle

Do not start with user interfaces or advanced engines. Start with the runtime foundation:

1. Sentinel Kernel
2. Universal Object Model
3. Event Bus

4. Evidence Records

5. Policy Stub

6. Trust Stub

7. Plugin Contract

8. Test Harness

Every later engine depends on these foundation modules.

Fixed repository hierarchy

The repository structure is immutable:

```
Project_Sentinel/
■■■ archive/
■■■ blueprint/
■■■ docs/
■■■ src/
■■■ tests/
■■■ tools/
■■■ build/
■■■ releases/
■■■ scripts/
■■■ assets/
■■■ examples/
■■■ plugins/
■■■ sdk/
■■■ api/
■■■ README.md
```

Implementation sequence

The project should be coded in milestones:

• C-001: Foundation Runtime

• C-002: Event, Evidence, Trust, and Policy Core

• C-003: Knowledge Graph and Mission Context

• C-004: Workflow and Automation Runtime

• C-005: Asset, Identity, and Machine Identity Engines

• C-006: Detection, Threat Intelligence, Vulnerability, and Exposure Engines

• C-007: Incident, Forensics, Compliance, Risk, and Assurance Engines

• C-008: AI Governance, Privacy, Data Protection, and Platform Hardening

• C-009: API, SDK, Plugin Ecosystem, CLI, and Developer Tooling

• C-010: Deployment, Operations, Release, and Production Readiness

Recommended technical baseline

Use Python for the first implementation stage. Suggested baseline:

• Python 3.12+

• Pydantic for object schemas

• FastAPI for API services later

• pytest for testing

• structlog or standard JSON logging

• SQLite for local development, PostgreSQL for server deployment

• In-memory event bus first, pluggable backend later

• Docker only after the core runtime works locally

What Codex should do first

Codex should create the project skeleton, package metadata, test configuration, and the first foundation modules under `src/sentinel/`. The first goal is not production functionality; it is a clean, testable runtime foundation where Sentinel objects can be created, events can be published and consumed, evidence can be attached, and engines can be registered and started by the Kernel.